

Marais'R'Sense

Dossier technique – Partie individuelle



Benoît MARAIS
MENUISERIE

Étudiant	Matéo ROUSSEAU
Lycée	Lycée de l'Hyrôme – Chemillé
Entreprise	Menuiserie Benoît Marais – Valanjou (49)
Professeur responsable	Mr. SUAREZ
Année scolaire	2025 / 2026

Table des matières

Table des matières.....	2
1 – Situation dans le projet.....	3
1.1 – Synoptique de la réalisation.....	3
1.2 – Description de la partie personnelle.....	3
2 – Réalisation des fonctions et cas d'utilisation.....	4
2.1 – Fonction FS1/FT1 – Acquisition des particules fines (SDS011).....	4
2.1.1 – Présentation de la tâche et choix technologique.....	4
2.1.2 – Conception détaillée – Classe CapteurParticules.....	4
2.2 – Fonctions FS4/FS5/FT3.1/FT3.3/FT3.4 – Interface Homme-Machine (IHM).....	5
2.2.1 – Choix technologique.....	5
2.2.2 – Architecture des écrans.....	5
2.2.3 – Navigation et interaction physique.....	6
2.2.4 – Thread-safety et propriétés Kivy.....	7
2.3 – Fonctions FS6/FT5/FT5.1/FT5.3 – Communication MQTT sécurisée.....	8
2.3.1 – Architecture du client MQTT.....	8
2.3.2 – Structure des messages MQTT.....	8
2.3.3 – Gestion des erreurs réseau.....	8
2.4 – Architecture MVC – Classe Controller.....	9
2.4.1 – Rôle du Controller.....	9
2.6 – Tests unitaires.....	10
2.6.1 – Récapitulatif des tests prévus et réalisés.....	10
2.6.2 – Test unitaire du module MQTTClient.....	10
2.6.3 – Problèmes rencontrés.....	11
3 – Bilan de la réalisation personnelle.....	12
3.1 – Points validés.....	12
3.2 – Améliorations possibles.....	12

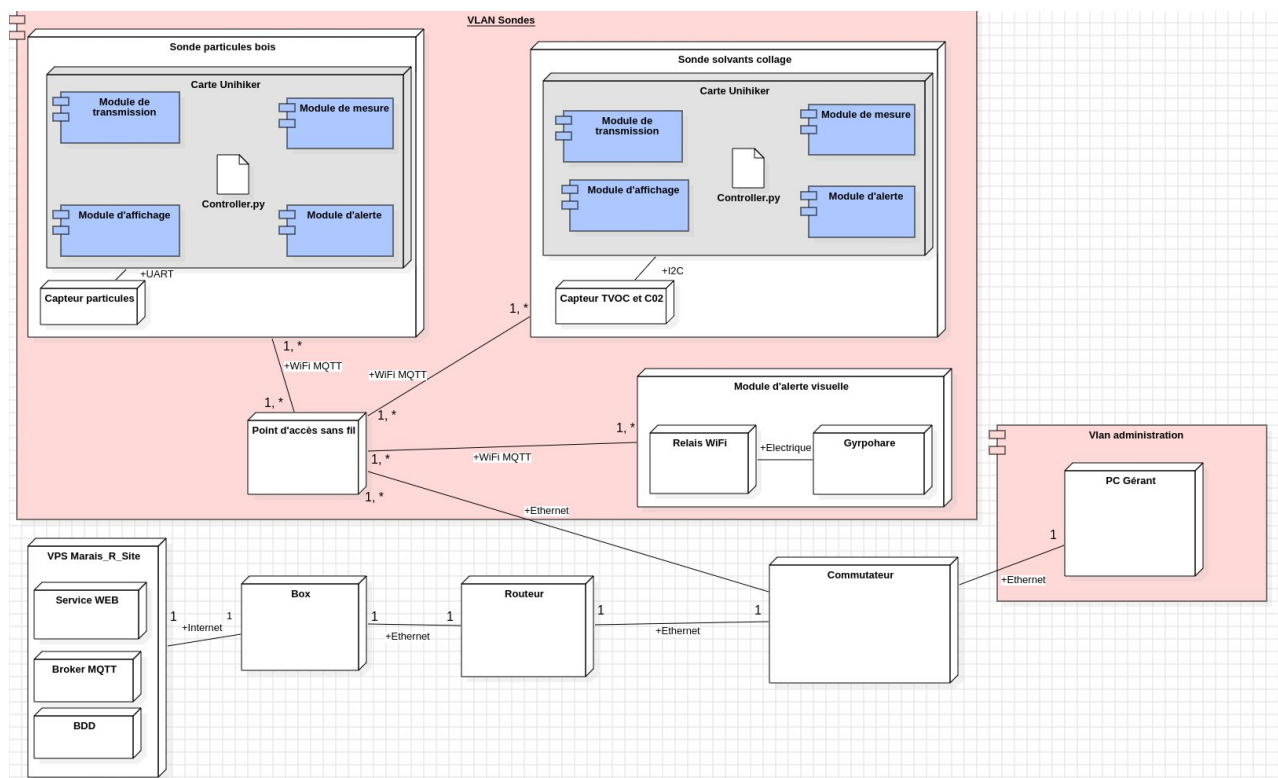
1 – Situation dans le projet

1.1 – Synoptique de la réalisation

Le projet Marais'R'Sense s'inscrit dans une architecture IoT composée de plusieurs briques distinctes. En tant qu'étudiant 1, j'ai la mission de configurer et exploiter la sonde de mesure des particules fines (capteur SDS011 / PM10), créer l'interface homme-machine (IHM Kivy) ainsi qu'utiliser la communication MQTT sécurisée vers le broker distant pour envoyer les mesures.

La sonde sur laquelle j'interviens sera déployée à proximité des machines de coupe dans l'atelier principal. Mon travail couvre le logiciel embarqué de cette sonde (couches Model, View et Controller du principe MVC).

Diagramme de déploiement :



1.2 – Description de la partie personnelle

Les objectifs de ma réalisation sont les suivants :

- Acquérir les mesures PM2.5 et PM10 depuis le capteur laser SDS011 via une liaison série UART, avec gestion de la reconnexion automatique en cas de débranchement.
- Afficher ces mesures en temps réel sur l'écran tactile 2,8" de la carte Unihiker, en indiquant visuellement (vert/orange/rouge) le niveau de risque par rapport aux seuils réglementaires.
- Publier chaque mesure vers le broker MQTT du projet (marais2026.btssn.ovh) via une connexion TLS sécurisée, dans un format JSON normalisé partagé avec l'équipe.
- Recevoir dynamiquement les seuils configurés depuis le site web Marais'R'Site et mettre à jour l'affichage sans redémarrage.
- Afficher l'identification de la sonde (adresse IP et MAC) sur une page dédiée de l'IHM.
- Configurer la segmentation VLAN sur le commutateur de niveau 2 pour isoler les sondes du réseau administratif.

2 – Réalisation des fonctions et cas d'utilisation

2.1 – Fonction FS1/FT1 – Acquisition des particules fines (SDS011)

2.1.1 – Présentation de la tâche et choix technologique

La tâche consiste à interfacer le capteur laser SDS011 avec la carte Unihiker afin de mesurer les concentrations de particules fines PM2.5 et PM10 dans l'air de l'atelier. Le SDS011 utilise une communication série UART à 9600 bauds et un protocole propriétaire documenté dans sa datasheet.

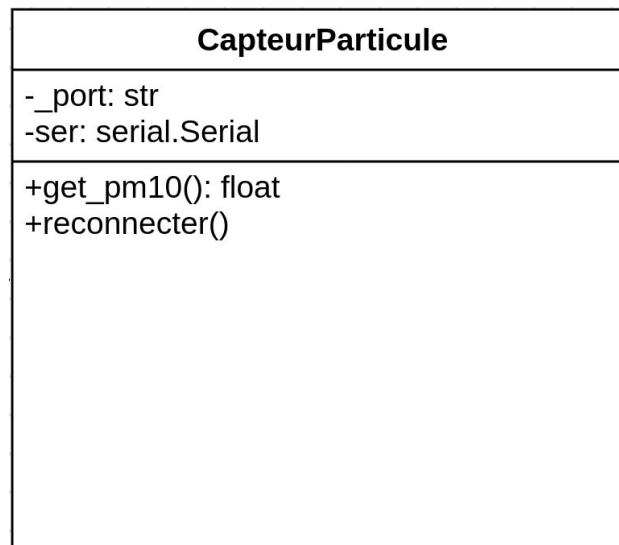
Choix de la bibliothèque : la bibliothèque Python open-source sds011 (py-sds011) a été retenue car elle abstrait le protocole série et propose nativement un mode query permettant de ne demander une mesure que lorsque c'est nécessaire, ce qui permet de mettre le laser en veille entre deux mesures et de prolonger sa durée de vie qui est de 8000 heures.

La classe CapteurParticules hérite de SDS011 (héritage de spécialisation) et ajoute :

- Une méthode `get_pm10()` orchestrant la séquence réveil → stabilisation → mesure → veille.
- Une méthode `reconnecter()` pour traiter le débranchement à chaud du capteur USB.

2.1.2 – Conception détaillée – Classe CapteurParticules

Diagramme de classe simplifié (extrait du diagramme complet présenté en Revue 2) :



Description des méthodes principales :

- **get_pm10()** : méthode de mesure principale appelée par la boucle du Controller. Elle appelle `sleep(False)` pour réveiller le laser, attend 30 secondes de stabilisation (valeur recommandée par la documentation), exécute `query()` pour lire PM2.5 et PM10, remet le capteur en veille puis attend 89 secondes. La durée totale d'un cycle est donc ≈ 2 minutes. Si `query()` retourne `None` (timeout série), une `SerialException` est levée et interceptée par le Controller qui appelle `reconnecter()`.
- **reconnecter()** : procédure bloquante à trois étapes – (1) fermeture du port pour libérer le verrou système, (2) attente active que `/dev/ttyUSB0` soit à nouveau accessible (re-branchement USB), (3) ré-invocation du constructeur parent SDS011 pour rétablir la connexion.

Extrait significatif – méthode get_pm10()

```
def get_pm10(self) -> float:
    # Appel sleep(False)
    for _ in range(2):
        self.sleep(sleep=False)

    # Stabilisation des mesures
    time.sleep(30)

    # Requête de mesure
    result = self.query()

    if result is None:
        # Aucune réponse dans le délai timeout : capteur probablement débranché
        raise serial.SerialException("Pas de réponse du capteur SDS011")

    _, pm10 = result  # on ignore pm2.5, on ne garde que pm10

    # Remise en veille du capteur pour économiser la durée de vie du laser
    self.sleep()
```

2.2 – Fonctions FS4/FS5/FT3.1/FT3.3/FT3.4 – Interface Homme-Machine (IHM)

2.2.1 – Choix technologique

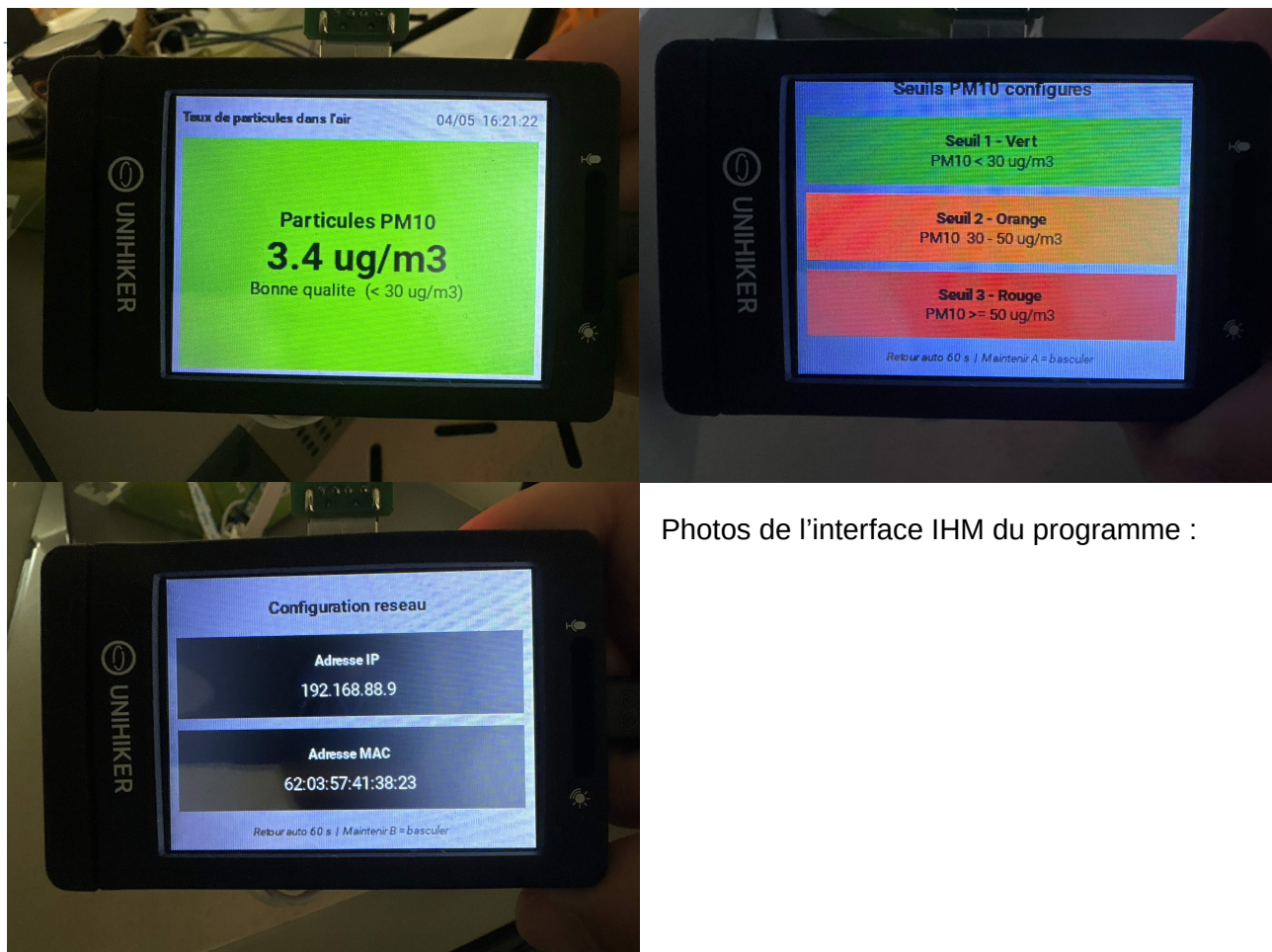
Le framework Kivy a été retenu pour l'IHM car il est nativement supporté par la carte Unihiker, il gère l'écran tactile et la rotation de l'affichage (240×320 px, rotation 90°), et il propose un système de liaison de propriétés particulièrement adapté pour mettre à jour l'affichage depuis un thread de fond.

L'architecture MVC est strictement respectée : la classe IHM (App Kivy) ne contient que de la logique de présentation, sans aucun accès aux capteurs ni à MQTT. Toutes les mises à jour graphiques passent par l'IHM.

2.2.2 – Architecture des écrans

L'IHM comporte six écrans gérés par un ScreenManager en mode NoTransition (pas d'animation, pour économiser les ressources du processeur) :

Nom écran	Classe Python	Rôle
'accueil'	AccueilScreen	Mesure PM10 en temps réel + code couleur.
'seuils'	SeuilsScreen	Affichage des seuils PM10 configurés (vert / orange).
'capteur2'	AccueilCapteur2Screen	Mesures TVOC et eCO2 (capteur étudiant 2, partagé).
'seuils_tvoc'	SeuilsTVOCScreen	Seuils TVOC configurés.
'seuils_co2'	SeuilsCO2Screen	Seuils eCO2 configurés.
'reseau'	ReseauScreen	Adresse IP et MAC de la sonde (identification).



Photos de l'interface IHM du programme :

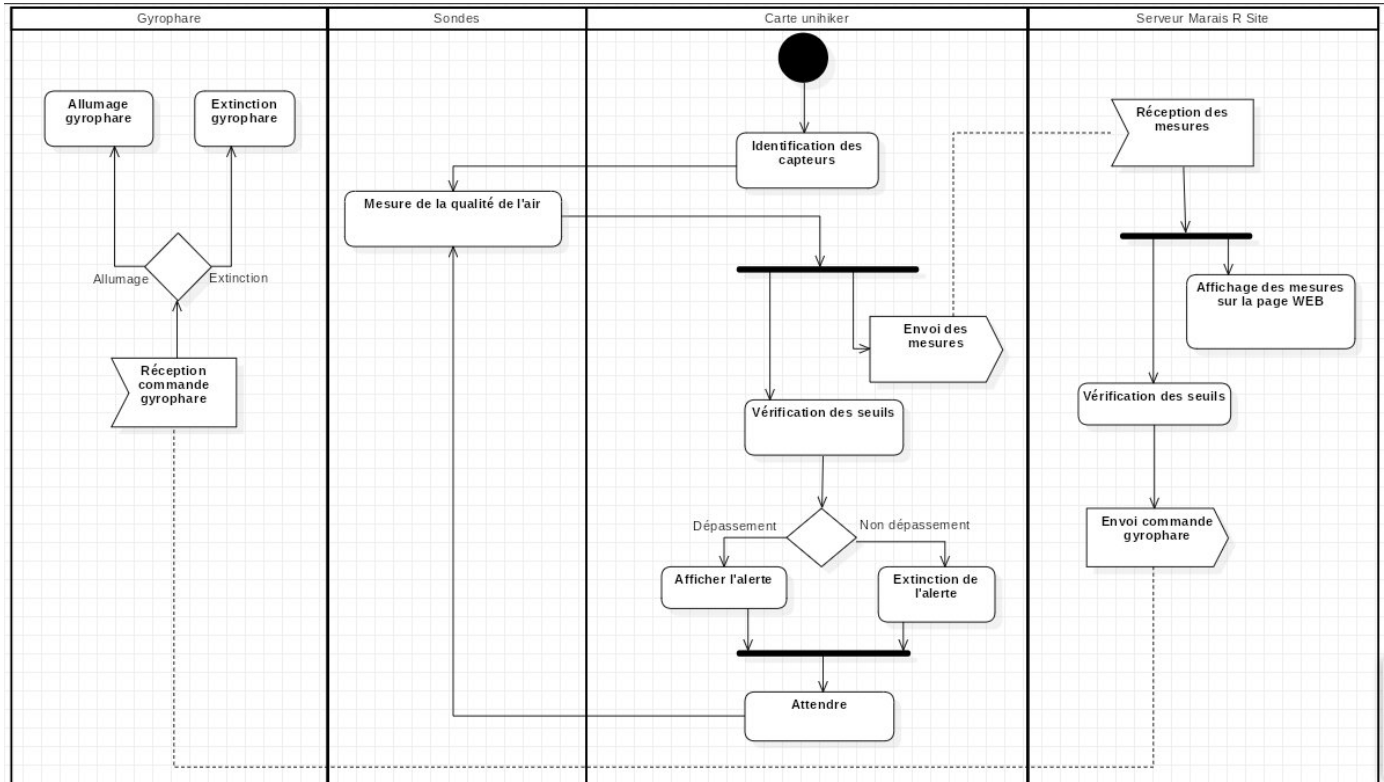
2.2.3 – Navigation et interaction physique

La carte Unihiker dispose de deux boutons physiques A et B. Leur lecture est réalisée par polling toutes les 100 ms via `Clock.schedule_interval()`. Un algorithme de détection de front montant mémorise l'horodatage du premier appui ; si la durée écoulée dépasse 3 s sans relâchement, l'action est déclenchée une seule fois :

- Bouton A : bascule entre le contexte « mesures » et le contexte « seuils ».
- Bouton B : affiche la page réseau (adresse IP, MAC) puis retour automatique après 60 s.

Le Controller orchestre également une alternance automatique entre les pages, toutes les 10 secondes, à l'aide de `threading.Timer` afin de présenter successivement les différentes mesures sans intervention de l'opérateur.

Diagramme d'activité :



2.2.4 – Thread-safety et propriétés Kivy

La bibliothèque Kivy impose que toute modification du graphe de scène soit effectuée depuis le thread principal. Les boucles de mesure s'exécutent dans des threads de fond (daemon=True). Toutes les méthodes de l'API publique de IHM utilisent `Clock.schedule_once(lambda dt: ..., 0)` pour repousser l'exécution dans le thread Kivy.

Les seuils sont stockés sous forme de `NumericProperty` Kivy (seuil_vert, seuil_orange, etc.). Dès qu'une valeur change, Kivy propage automatiquement la mise à jour vers tous les widgets liés dans le fichier KV (data binding réactif), sans code supplémentaire.

2.3 – Fonctions FS6/FT5/FT5.1/FT5.3 – Communication MQTT sécurisée

2.3.1 – Architecture du client MQTT

La classe MQTTClient encapsule la bibliothèque paho-mqtt. Un seul client est instancié par application afin d'éviter plusieurs connexions simultanées au broker. La connexion est maintenue en permanence par `loop_start()` qui démarre un thread réseau interne non bloquant.

La sécurité est assurée par :

- Chiffrement TLS (`tls_set` avec le certificat CA du broker, port 8883).
- Authentification par identifiant et mot de passe (`username_pw_set`).
- Ré-abonnement automatique au topic des seuils dans le callback `on_connect`, ce qui garantit qu'une reconnexion après coupure réseau restaure tous les abonnements.

2.3.2 – Structure des messages MQTT

Le format JSON des messages a été défini en concertation avec l'équipe (FT5.1) pour être homogène avec les données attendues par le serveur Marais'R'Site :

Topic	Payload JSON exemple
marais/sondes/<MAC>/pm10	{"timestamp": "2026-04-15T10:23:00", "mesure": {"PM10": 42.5}}
marais/sondes/<MAC>/tvoc_co2	{"timestamp": "...", "mesure": {"ECO2": 850, "TVOC": 320}}
marais/seuils/config (réception)	{"seuil_vert": 25.0, "seuil_orange": 50.0}

L'adresse MAC est calculée automatiquement via `uuid.getnode()` et insérée dans le topic, ce qui permet d'identifier chaque sonde sans configuration manuelle. Le format d'adresse retourné par `uuid.getnode()` est LSB en premier ; la correction de l'ordre des octets est effectuée dans le constructeur.

2.3.3 – Gestion des erreurs réseau

Si le broker est inaccessible au démarrage, le Controller affiche un popup dans l'IHM (« Broker MQTT inaccessible. Seuils par défaut actifs. ») et continue de fonctionner avec les seuils par défaut définis dans `IHM.py`. En cas d'échec de publication pendant la boucle de mesure, le Controller tente une réinitialisation du client MQTT (appel de `_init_mqtt()`).

2.4 – Architecture MVC – Classe Controller

2.4.1 – Rôle du Controller

Le Controller est le seul composant qui connaît à la fois le Model (capteurs, MQTT) et la View (IHM). Il est instancié par main.py et son point d'entrée public est prise_mesure_et_envoi(). Il assume les responsabilités suivantes :

- Détection automatique des capteurs disponibles au démarrage (_detecter_pm10, _detecter_tvoc_co2) ; l'écran initial affiché dépend des capteurs trouvés.
- Lancement des boucles de mesure dans des threads de fond daemon (threading.Thread).
- Navigation entre les écrans via l'API IHM.navigate_to(), avec gestion des contextes « mesures », « seuils » et « réseau ».
- Alternance automatique toutes les 10 s entre les pages (threading.Timer réarmé après chaque déclenchement).
- Retour automatique vers les mesures après 60 s d'inactivité depuis les pages seuils ou réseau.
- Transmission des seuils reçus via MQTT à l'IHM (callback _on_seuils_recus → ihm.update_seuils()).

2.6 – Tests unitaires

2.6.1 – Récapitulatif des tests prévus et réalisés

Les tests unitaires ont été planifiés pour les modules suivants. Les tests des modules partagés (MQTTClient, MesureTVOC_CO2, Ccs811) ont été menés conjointement avec l'étudiant 2. La rédaction formelle des rapports de test et les tests unitaires du module CapteurParticules et de l'IHM sont en cours de finalisation (état au moment de la Revue 3).

Module testé	Objectif	État	Commentaire
MQTTClient	Connexion broker, publication PM10, réception seuils	Terminé ✓	Testé avec broker de test Mosquitto local.
Ccs811	Scan I2C, HW_ID, lecture ALG_RESULT	Terminé ✓	Test mené avec étudiant 2.
MesureTVOC_CO2	Run-in, reconnexion, get_mesures()	Terminé ✓	Test mené avec étudiant 2.
CapteurParticules	get_pm10(), reconnecter()	En cours (70 %)	Test de reconnexion à finaliser.
IHM – AccueilScreen	Changement couleur selon seuils, horloge	En cours (90 %)	Test sous Kivy desktop en cours.

2.6.2 – Test unitaire du module MQTTClient

Ce test a été réalisé et est représentatif de la démarche appliquée à l'ensemble des modules.

Élément testé	MQTTClient – publish_pm10(), subscribe_seuils()
Objectif du test	Vérifier la connexion TLS au broker, la publication d'un message PM10 et la réception des seuils via callback.
Nom du testeur	Matéo ROUSSEAU Date : semaine 16 (avril 2026)
Moyens mis en oeuvre	Logiciel : Python 3, paho-mqtt, Mosquitto (broker local test) Matériel : PC Linux Outil : pytest + MQTT Explorer
Procédure	- Démarrer un broker Mosquitto local sans TLS (test simplifié). - Instancier MQTTClient avec broker='localhost', port=1883. - Appeler publish_pm10(42.5) et vérifier la réception dans MQTT Explorer. - Publier sur le topic seuils, vérifier que le callback est appelé avec les bonnes valeurs.

Id	Vecteur de test	Résultat attendu	Résultat obtenu	Validation
T1	Connexion broker local (sans TLS)	on_connect rc=0, log 'MQTT connecte'	rc=0 constaté dans les logs	O
T2	publish_pm10(42.5)	Message JSON reçu dans MQTT Explorer avec PM10:42.5 et timestamp ISO	Conforme, message reçu	O
T3	Publier JSON seuils {seuil_vert:25,	Callback appelé avec sv=25.0, so=50.0	Callback appelé, valeurs correctes	O

	seuil_orange:50}			
T4	Publier JSON seuils malformé {toto:1}	KeyError intercepté, log d'erreur, pas de crash	Exception capturée, programme continue	O
T5	Broker inaccessible au démarrage	socket.error levée, Controller affiche popup	Exception propagée correctement	O

Conclusion du test

Tous les cas sont validés. La connexion TLS avec certificat CA sera testée sur le broker de production lors de la phase d'intégration (test d'intégration T-INTEG-01).

2.6.3 – Problèmes rencontrés

Les problèmes significatifs rencontrés et les solutions mises en oeuvre sont résumés ci-dessous :

N°	Problème	Cause identifiée	Solution
1	Double affichage des alertes Kivy au démarrage (notifications côte à côte)	Kivy chargeait ihm.kv automatiquement (convention de nommage) puis build() le rechargeait une deuxième fois.	Surcharge de load_kv() pour désactiver le chargement automatique et ne charger le fichier KV qu'une seule fois explicitement dans build().
2	query() retournait None sans lever d'exception, laissant la boucle tourner en silence.	La bibliothèque sds011 retourne None en cas de timeout au lieu de lever une exception.	Ajout d'un test explicite sur None dans get_pm10() et levée d'une SerialException pour déclencher la procédure de reconnexion.
3	Ordre des octets inversé dans l'adresse MAC calculée par uuid.getnode()	uuid.getnode() renvoie un entier avec le LSB en premier.	Correction du calcul : itération de i=5 à i=0 pour reconstruire l'adresse dans l'ordre conventionnel (MSB en premier).
4	Bouton physique déclenché plusieurs fois pour un seul appui (rebond logiciel)	Intervalle de polling de 1 ms trop court	Réduction du polling à 100 ms avec anti-rebond par horodatage (time.monotonic()).

3 – Bilan de la réalisation personnelle

3.1 – Points validés

À la date de la Revue 3, les éléments suivants sont fonctionnels et ont été validés :

- Acquisition PM10 depuis le capteur SDS011 avec gestion de la reconnexion automatique (FS1 / FT1).
- Affichage temps réel sur l'IHM Unihiker avec code couleur vert/orange/rouge selon les seuils (FT3.1 / FS5).
- Affichage de l'adresse IP et MAC sur la page réseau (FT3.3).
- Réception des seuils configurés via MQTT et mise à jour dynamique de l'IHM sans redémarrage (FT3.4 / FT5.3).
- Publication des mesures PM10 en JSON horodaté vers le broker MQTT sécurisé TLS (FT5 / FT5.1 / FS6).
- Navigation entre les écrans par les boutons A/B avec maintien 3 s et alternance automatique 10 s (FT3.4).
- Tests unitaires MQTTClient, Ccs811, MesureTVOC_CO2 terminés (partagés avec étudiant 2).
- Maquette VLAN Cisco sous Packet Tracer documentée (FT6 / FT8).

3.2 – Améliorations possibles

- Implémentation de tests unitaires automatisés avec pytest et unittest.mock pour simuler le port série et le broker MQTT sans matériel réel, ce qui faciliterait les tests en environnement de développement.
- Mise en place d'un watchdog logiciel (threading.Timer) qui redémarre les threads de mesure s'ils se terminent de manière inattendue.
- Création de boîtes pour contenir et fixer les différents